

强化学习核心算法笔记

第一部分：TD(0) 与蒙特卡洛 (Monte Carlo)

一、TD(0) 更新公式

$$V(S_t) \leftarrow V(S_t) + \alpha [R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

逐项拆解

符号	含义
$V(S_t)$	当前对状态 S_t 价值的估计
α	学习率 (步长), 控制每次更新迈多大步
R_{t+1}	执行动作后 实际拿到 的即时奖励
$\gamma V(S_{t+1})$	对下一步 状态价值 的折扣估计
$R_{t+1} + \gamma V(S_{t+1})$	目标值 : 当前认为”应该值多少”
$V(S_t)$ (方括号内)	原来的估计值
$R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$	TD 误差 : 目标值与原估计的偏差

核心含义 用”下一个状态的估计值”来更新当前状态。

方括号里的目标值 $R_{t+1} + \gamma V(S_{t+1})$ 是**当前对真实回报的猜测**: 先拿即时奖励 R_{t+1} , 再加上对后续价值的估计 $\gamma V(S_{t+1})$ 。

如果目标值比原估计大, 说明当前状态比之前想的更好, 就往上调; 反之往下调。调整幅度由 TD 误差和学习率共同决定。

关键特征: $V(S_{t+1})$ 本身也是一个估计值 (可能不准), 所以 TD 是在**用估计去修正估计**, 这叫**自举 (bootstrapping)**。

二、蒙特卡洛 (MC) 更新公式

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

逐项拆解

符号	含义
G_t	实际回报 : 从状态 S_t 开始, 到轨迹终止, 所有奖励的累积和 (含折扣)
$G_t - V(S_t)$	回报误差 : 真实发生的总回报与原估计的偏差

核心含义 用”实际发生的总回报”来更新当前状态。

G_t 是这条轨迹上**真实采样出来的完整回报**, 不是估计。等整条轨迹跑完后, 把实际拿到的总分和原来的估计对比, 差多少就修正多少。

关键特征: MC 不依赖 $V(S_{t+1})$ 这种中间估计, 而是等到终止状态拿到**真实累积回报**后才更新。没有自举, 没有”估计套估计”。

三、两者的本质差异

	TD(0)	蒙特卡洛
目标值来源	$R_{t+1} + \gamma V(S_{t+1})$, 基于下一步的估计	G_t , 基于整条轨迹的真实采样
是否需要等终止	不需要, 每步都能更新	必须等轨迹结束
偏差来源	有偏差: $V(S_{t+1})$ 本身不准	无偏差: G_t 是真实回报
方差来源	方差小: 只依赖一步随机性	方差大: 依赖整条轨迹的随机性
核心操作	自举 (bootstrapping)	采样平均

四、直观对比 (以具体轨迹为例)

轨迹: $S_1 \xrightarrow{r=1} S_2 \xrightarrow{r=2} S_3 \xrightarrow{r=3} \text{终止}$

TD 更新 S_1 时:

- 目标值 = $1 + V(S_2)$, 但此时 $V(S_2)=0$ (初始值)
- 所以 TD 认为 S_1 的目标值只有 1, 更新后 $V(S_1)=0.5$
- 问题: TD 不知道 S_2 后面其实还有 $2+3=5$ 的奖励, 因为它还没走到那

MC 更新 S_1 时:

- 目标值 = $G_1 = 1+2+3 = 6$ (整条轨迹跑完后的真实总分)
- 所以 MC 直接告诉 S_1 : 你这条轨迹实际值 6
- 更新后 $V(S_1)=3$

TD 的 S_1 值偏小, 是因为它依赖 $V(S_2)$ 的初始估计 (0), 而这个估计不准。随着后续继续采样, $V(S_2)$ 会被修正, TD 的 S_1 也会逐步趋近真实值。

MC 一步到位, 但需要等轨迹结束; TD 步步为营, 但早期依赖不准确的自举值。

第二部分: SARSA 与 Q-Learning

一、本质定位

两者都是时序差分 (TD) 控制算法, 目标都是学习最优状态-动作值函数 $Q(s, a)$ 。核心差异在于更新时如何处理下一步动作。

二、SARSA (State-Action-Reward-State-Action)

更新公式

$$Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$$

五个字母的含义 更新一次需要连续 5 个变量: 当前状态 S 、当前动作 A 、获得的奖励 R 、下一状态 S' 、在 S' 实际执行的动作 A' 。

关键机制 公式中的 A' 是智能体在 S' 实际选择的动作, 不是理论上最好的动作。

这意味着 SARSA 的更新目标依赖于实际行为轨迹。

策略关系：On-policy（同策略） SARSA 学习的是当前正在执行的策略。

如果当前用 ϵ -greedy 探索（有 ϵ 概率随机乱走），SARSA 就假设未来也会继续用 ϵ -greedy 乱走，更新时把这点考虑进去。

三、Q-Learning

更新公式

$$Q(S,A) \leftarrow Q(S,A) + \alpha \left[R + \gamma \max_{a'} Q(S',a') - Q(S,A) \right]$$

关键机制 公式中的 $\max_{a'} Q(S',a')$ 是在 S' 所有动作中取 **Q 值最大的那个**，完全不管智能体在 S' 实际做了什么。

策略关系：Off-policy（异策略） Q-Learning 学习的是**最优贪婪策略**（总是选 Q 最大的动作），但智能体实际探索时可以用另一个策略（比如 ϵ -greedy）。

行为策略（实际探索用的）和**目标策略**（学习用的最优策略）是分离的。

四、核心差异对比

	SARSA	Q-Learning
目标值	$R + \gamma Q(S',A')$	$R + \gamma \max_{a'} Q(S',a')$
A' 的来源	实际执行的动作	假设最优的动作（取 max）
策略关系	On-policy: 学习的就是实际执行的策略	Off-policy: 学习最优策略，实际可以乱探索
对探索的态度	把探索风险算进价值估计	不考虑探索风险，只假设未来最优
更新方向	保守（考虑实际可能踩坑）	激进（直接逼近理论最优）

五、经典例子：悬崖行走（Cliff Walking）

一个网格任务，边缘是悬崖（掉下去大惩罚），目标是走到终点。

- **最短路径**：贴着悬崖边走，步数最少。
- **ϵ -greedy 探索**：有 ϵ 概率随机乱走，贴着悬崖时可能掉下去。

SARSA 的行为：

更新时 $Q(S',A')$ 用的是实际动作（ ϵ -greedy 可能选到”掉悬崖”），所以 SARSA 认为”这个状态很危险”，学会**绕远路避开悬崖**。

Q-Learning 的行为：

更新时 $\max_{a'} Q(S',a')$ 假设在 S' 一定选最优动作（不会掉悬崖），所以 Q 值认为**贴边走最优**。但实际执行 ϵ -greedy 时，智能体仍然可能随机掉下去，**实际表现反而更差**。

六、具体题目中的数值对比

	SARSA	Q-Learning
S' 上的动作选择	实际选了 a_1 , $Q = 3$	不管实际选了什么, 取最大 $Q = 5$
目标值	$1 + 0.9 \times 3 = 3.7$	$1 + 0.9 \times 5 = 5.5$
结果	$Q(S, a_1) = 2.17$	$Q(S, a_1) = 2.35$

SARSA 更新更小, 因为它认为智能体在 S' 实际走了 a_1 (价值 3), 后续也就值 3; Q-Learning 认为如果最优的话应该走 a_2 (价值 5), 所以把 $Q(S, a_1)$ 往更高的方向推。

一句话总结: SARSA 跟着实际行为走 (胆小保守), Q-Learning 按最优假设走 (激进理想化)。